

findings

Semgrep Onboarding Findings — §11.4.166

Date: 2026-06-22 **Scope:** Install + validate + real-scan semgrep per constitution §11.4.166 (Universal Semgrep static analysis mandate). Wiring edits (.mcp.json / shell rc / pre-commit hook) deliberately NOT applied — left to the conductor's meta-repo commit. **Working dir:** /Volumes/T7/Projects/helix_code

1. Constitution scripts — what each does

constitution/scripts/semgrep/ (POSIX-sh, §11.4.67-compliant):

- **semgrep_setup.sh** — install/build integration. `semgrep_setup_check()` returns 0 if semgrep on PATH + functional; `semgrep_setup_install()` does `pip install --user semgrep` (falls back `pip3/pip`), re-checks `~/ .local/bin/ /usr/local/bin/ ~/ .cargo/bin`, anti-bluff verifies semgrep `--version` non-empty after (install exit 0 != working binary); `semgrep_setup_build()` builds from the submodules/semgrep OCaml source when toolchain present, else falls back to pip. Logs events to `constitution/docs/semgrep/Status.md`. Exit codes 0=ok, 1=bluff-caught, 2=env problem.
- **semgrep_path.sh** — sourced from `.bashrc/.zshrc`. Prepends `~/ .local/bin, ~/ .cargo/bin, /usr/local/bin, /opt/homebrew/bin, ~/bin` to PATH (dedup via case match), exports PATH. Does not require semgrep pre-installed.
- **semgrep_validate.sh** — anti-bluff validation (§107/§11.4.69). `semgrep_validate_check()` runs `semgrep scan --config auto --json` on a generated C buffer-overflow fixture + asserts valid JSON with “results” (python3 json.tool, grep fallback), writes evidence JSON; `semgrep_validate_rules()` runs `semgrep --config-registry --dump-config-rules | head -5` as a registry-reachability probe. Emits `ab_pass_with_evidence / ab_fail` lines + logs to `Status.md`. Exit 0=PASS,1=FAIL,2=env.
- **semgrep_ci_test.sh** — CI integration test, evidence under `qa-results/<run-id>/`. 4 tests: (1) semgrep on PATH, (2) `--version` non-zero, (3) known-vuln detection (Python `eval(input())` fixture, checks JSON `check_id` count `>=1`, cross-checks with `--error` exit 2), (4) shell-script scan

of scripts/ *.sh (no-crash, exit 0 or 2). Records PASS/FAIL/SKIP counts; exit 1 if any FAIL.

2. Is semgrep installed?

YES — already installed before this session.

```
$ semgrep --version
1.167.0
$ command -v semgrep
/opt/homebrew/bin/semgrep      (Homebrew, not pip --user)
```

No install needed. semgrep --version confirms working binary (not just a present file).

3. semgrep_validate.sh verdict

```
$ sh constitution/scripts/semgrep/semgrep_validate.sh
PASS: semgrep scan produced valid JSON with results [evidence: constitution]
PASS: semgrep registry reachable (dump-config-rules returned output) [evidence: constitution]
=== semgrep validation: ALL PASS ===
(exit 0)
```

4. Real scan evidence — helix_code/internal/auth + helix_code/internal/llm

```
$ semgrep scan --config auto helix_code/internal/auth helix_code/internal/llm
Scanning 112 files tracked by git with 1059 Code rules (Community registry)
  <multilang> 47 rules / 112 files ; go 84 / 100 ; yaml 31 / 2
Ran 161 rules on 112 files: 4 findings.
Scan completed successfully. Findings: 4 (4 blocking). Rules run: 161. Tarball generated.
(exit 0 - semgrep OSS exits 0 even with findings; use --error for non-zero)
```

SEMGREP_APP_TOKEN absent → registry/Community rulepacks used (Pro/app tier honestly SKIPPED per §11.4.3; --config auto works without a token). Full output: scan_auth_llm.txt (sibling).

4 findings — ALL go.lang.security.audit.dangerous-exec-command (Blocking), all in helix_code/internal/llm/:

- auto_llm_manager.go:520 — cmd := exec.Command("bash", "-c", script)

- `local_llm_manager.go:439` — `cmd := exec.CommandContext(ctx, "bash", "-c", provider.BuildScript)`
- `model_converter.go:303` — `cmd := exec.CommandContext(ctx, tool.Command, job.Args...)`
- `model_download_manager.go:479` — `cmd := exec.Command(tool.Command, args...)`

Rule: non-static command in `exec.Command` — audit input for code-injection if unverified user data reaches the call site (<https://sg.run/W8lA>). These are audit-class findings (not confirmed exploits); each needs an input-provenance audit. NOTE these are the real `exec` paths the constitution's anti-bluff BLUFF-003 demanded (real `os/exec`, not simulated) — the finding is about input trust, not about the `exec` being a bluff.

5. WIRING STILL NEEDED (NOT applied — for the conductor)

All four §11.4.166 wiring points are currently ABSENT:



.mcp.json “semgrep” MCP server — present servers: `codegraph`, `media-validator`, `open-design`. `semgrep` NOT present. Add a `semgrep mcpServers` entry (per §11.4.78 step-3 `CodeGraph` pattern; the `semgrep` MCP plugin is available in this environment as `mcp__plugin_semgrep_semgrep__`).



.bashrc/.zshrc PATH integration — neither `~/.zshrc` nor `~/.bashrc` source `constitution/scripts/semgrep/semgrep_path.sh`. (Currently moot since `semgrep` is on `PATH` via Homebrew, but §11.4.166(2) requires the `rc` source line for all-users/all-shells availability.)



pre-commit semgrep scan hook — `.git/hooks/pre-commit` exists but has NO `semgrep` reference. Reference hook already shipped at `constitution/scripts/hooks/semgrep_precommit.sh` (executable) — wire it into the installed pre-commit (or `scripts/install_git_hooks.sh`) to run `semgrep scan --config auto --error` (blocks commit/push on any finding) per §11.4.166(3).



docs_chain context — `.docs_chain/contexts/semgrep_status.yaml` absent (§11.4.166(6)).

Scripts are inherited by reference from the constitution submodule per §11.4.28/§11.4.35 — wire, do not copy.

Sources verified 2026-06-22

- constitution/Constitution.md §11.4.166 (Universal Semgrep static analysis mandate)
- semgrep 1.167.0 --config auto Community registry behaviour (observed this session)